

Operators

Operators can be easily defined as characters that represent an operation. These symbols perform different operations on several variables and values.

Example : $5 + 6 = 11$. Here, 5 and 6 are the **operands**, and + is called the **operator**.

Categories of Operators

Unary operators : perform an action with a **single operand**.

Binary operators: perform actions with two **operands**.

Types of Operator

1. Arithmetic Operator :

Arithmetic operator can be divided into two categories -

- **Binary Operators :** +, -, *, /(int/int will always yield int) , %
(Return remainder after dividing two numbers & with int (works perfectly) but with float (produces ambiguity)).

Special powers of / & % by powers of 10

/ : to reduce the number by 1 digit

% : to get last digit(s) of number

- **Unary Operators :**

Increment Operator (**++**) : Increase the value by 1.

Decrement Operator (**--**) : Decrease the value by 1.

+, **-** and **!**(DTL) is also a unary operator(-5, 5 (is same as +5)).

RULES for Increment and Decrement :

- Cannot applied to constant

Example : `int c = ++10; // compile-time error`

- Nesting of both operators is not allowed

Example : `int a = 10;`

`int b = ++(++a); // compile-time error [++11]`

- They are not operated over final variables

Example : `final int a = 10;`

`int b = ++a; // compile-time error`

- Increment and Decrement Operators can not be applied to booleans.

Example : `boolean a = false;`

`a++; // compile-time error`

Quiz On Increment And Decrement Operators :

<https://sheryians.com/whizz/test/651ce22ebf489c724c896167>

2. Relational Operators : Used to check the relations between two operands. They return a boolean value (true or false) by comparing the two operands.

Greater Than (>) ,Less Than (<), <=, >=, ==, !=

- **Equal To (==)** - Checks if two operands are equal
- **Not Equal To (!=)** - Checks if two operands are not equal
- **Greater Than or Equal To (>=)** - Checks if one operand is either greater than or equal to the other.
- **Less Than or Equal To (<=)** - Checks if one operand is either less than or equal to the other.

3. Logical operators : Combine multiple conditional statements. There are three types of logical operators in Java: **AND(&&)**, **OR (||)** and **NOT(!)** operators.

- **Logical AND Operator(&&)**

Returns **true** when both conditions under evaluation are true, otherwise it returns **false**.

e.g : `if(a>b && a<c) System.out.println("Maximum: " + a);`

- **Logical OR Operator(||)**

Returns **true** if any one of the given conditions is true, otherwise it returns **false**. It returns **false** if and only if **both conditions** under evaluation are false.

e.g : `if(a>b || a<c) System.out.println("Max: " + a);`

- **Logical Not Operator(!)**

It accepts a single value as an input and returns the inverse of the same. This is a unary operator unlike the AND and OR operators.

e.g : `if(!(a<b)) System.out.println("Max: " + a);`

4. ShortHand operators

The assignment operator can be combined with other operators to build a shorter version of the statement. **+=, -=, *=, /=, %=**

Example : `a = a+5`, we can write `a += 5`.

do not use **+= & -=** [(=) followed by a unary plus (+)]